

```
jakka_bhanu@LAPTOP-DCEV:~$ cd ~/skill1
jakka_bhanu@LAPTOP-DCEVSCQ0:~/skill1$ ls -la
total 56
drwxr-xr-x  6 jakka_bhanu jakka_bhanu 4096 Aug  1 10:14 .
drwxr-xr-x 13 jakka_bhanu jakka_bhanu 4096 Aug 27 05:28 ..
drwxr-xr-x  6 jakka_bhanu jakka_bhanu 4096 Aug  1 10:04 .git
-rw-r--r--  1 jakka_bhanu jakka_bhanu  26 Aug  1 10:05 .gitignore
-rw-r--r--  1 jakka_bhanu jakka_bhanu 178 Aug  1 10:06 Makefile
-rw-r--r--  1 jakka_bhanu jakka_bhanu   0 Aug  1 10:05 README.md
drwxr-xr-x  2 jakka_bhanu jakka_bhanu 4096 Aug  1 10:05 include
-rwxr-xr-x  1 jakka_bhanu jakka_bhanu 20512 Aug  1 10:14 myshell
drwxr-xr-x  2 jakka_bhanu jakka_bhanu 4096 Aug  1 10:05 obj
drwxr-xr-x  2 jakka_bhanu jakka_bhanu 4096 Aug  1 10:05 src
jakka_bhanu@LAPTOP-DCEVSCQ0:~/skill1$ cat src/main.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_ARGS 64

void parse_input(char *input, char **args) {
    int i = 0;
    char *token = strtok(input, " ");
    while (token != NULL && i < MAX_ARGS - 1) {
        args[i++] = token;
        token = strtok(NULL, " ");
    }
    args[i] = NULL;
}

int main() {
```

```
jakka_bhanu@LAPTOP-DCEVSCQ0:~/skill1$ cat src/main.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_ARGS 64

void parse_input(char *input, char **args) {
    int i = 0;
    char *token = strtok(input, " ");
    while (token != NULL && i < MAX_ARGS - 1) {
        args[i++] = token;
        token = strtok(NULL, " ");
    }
    args[i] = NULL;
}

int main() {
    char input[1024];
    char *args[MAX_ARGS];

    while (1) {
        printf("myshell> ");
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL) {
            printf("\n");
            break;
        }

        input[strcspn(input, "\n")] = '\0';
```

```
input[strcspn(input, "\n")] = '\0';

if (strlen(input) == 0) {
    continue;
}

if (strcmp(input, "exit") == 0) {
    break;
}

parse_input(input, args);

if (args[0] == NULL) {
    continue;
}

if (strcmp(args[0], "cd") == 0) {
    if (args[1] == NULL) {
        fprintf(stderr, "cd: expected argument\n");
    } else {
        if (chdir(args[1]) != 0) {
            perror("cd failed");
        }
    }
    continue;
}

pid_t pid = fork();

if (pid < 0) {
    perror("fork failed");
} else if (pid == 0) {
```

```
jakka_bhanu@LAPTOP-DCEVSCQ0:~/skill1$ nano src/main.c
}
    continue;
}

pid_t pid = fork();

if (pid < 0) {
    perror("fork failed");
} else if (pid == 0) {
    execvp(args[0], args);
    perror("exec failed");
    exit(1);
} else {
    wait(NULL);
}
}

return 0;
}jakka_bhanu@LAPTOP-DCEVSCQ0:~/skill1$ ./myshell
myshell> pwd
/home/jakka_bhanu/skill1
myshell> ls
Makefile  README.md  include  myshell  obj  src
myshell> exit
jakka_bhanu@LAPTOP-DCEVSCQ0:~/skill1$ nano src/main.c
jakka_bhanu@LAPTOP-DCEVSCQ0:~/skill1$
jakka_bhanu@LAPTOP-DCEVSCQ0:~/skill1$ #include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_ARGS 64
```

```
#define MAX_ARGS 64

void parse_input(char *input, char **args) {
    int i = 0;
    char *token = strtok(input, " ");
    while (token != NULL && i < MAX_ARGS - 1) {
        args[i++] = token;
        token = strtok(NULL, " ");
    }
    args[i] = NULL;
}

int main() {
    char input[1024];
    char *args[MAX_ARGS];

    while (1) {
        printf("myshell> ");
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL) {
            printf("\n");
            break;
        }

        input[strcspn(input, "\n")] = '\0';

        if (strlen(input) == 0) {
            continue;
        }

        if (strcmp(input, "exit") == 0) {
            break;
        }
    }
}
```

```
    }

    parse_input(input, args);

    if (args[0] == NULL) {
        continue;
    }

    if (strcmp(args[0], "cd") == 0) {
        if (args[1] == NULL) {
            fprintf(stderr, "cd: expected argument\n");
        } else {
            if (chdir(args[1]) != 0) {
                perror("cd failed");
            }
        }
        continue;
    }

    pid_t pid = fork();

    if (pid < 0) {
        perror("fork failed");
    } else if (pid == 0) {
        execvp(args[0], args);
        perror("exec failed");
        exit(1);
    } else {
        wait(NULL);
    }
}
```

```
return 0;
}jakka_bhanu@LAPTOP-DCEVSCQ0:~/skill1$ ./myshell
myshell> pwd
/home/jakka_bhanu/skill1
myshell> ls
Makefile README.md include myshell obj src
myshell> exit
jakka_bhanu@LAPTOP-DCEVSCQ0:~/skill1$ nano src/main.c
jakka_bhanu@LAPTOP-DCEVSCQ0:~/skill1$
jakka_bhanu@LAPTOP-DCEVSCQ0:~/skill1$ #include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_ARGS 64

void parse_input(char *input, char **args) {
    int i = 0;
    char *token = strtok(input, " ");
    while (token != NULL && i < MAX_ARGS - 1) {
        args[i++] = token;
        token = strtok(NULL, " ");
    }
    args[i] = NULL;
}

int main() {
    char input[1024];
    char *args[MAX_ARGS];

    printf("Shell started. My PID = %d, Parent (terminal) PID = %d\n", getpid(), getppid());
```

```
printf("Shell started. My PID = %d, Parent (terminal) PID = %d\n", getpid(), getppid());
```

```
while (1) {
    printf("myshell> ");
    fflush(stdout);

    if (fgets(input, sizeof(input), stdin) == NULL) {
        printf("\n");
        break;
    }

    input[strcspn(input, "\n")] = '\0';

    if (strlen(input) == 0) {
        continue;
    }

    if (strcmp(input, "exit") == 0) {
        break;
    }

    parse_input(input, args);

    if (args[0] == NULL) {
        continue;
    }

    if (strcmp(args[0], "cd") == 0) {
        if (args[1] == NULL) {
            fprintf(stderr, "cd: expected argument\n");
        } else {
            if (chdir(args[1]) != 0) {
```

```
                fprintf(stderr, "cd: expected argument\n");
            } else {
                if (chdir(args[1]) != 0) {
                    perror("cd failed");
                }
            }
            continue;
        }

        pid_t pid = fork();

        if (pid < 0) {
            perror("fork failed");
        }
        else if (pid == 0) {
            // ---- CHILD PROCESS ----
            printf("[CHILD] PID = %d, Parent PID = %d, about to run '%s'\n",
                getpid(), getppid(), args[0]);
            execvp(args[0], args);
            // execvp only returns if it failed
            perror("exec failed");
            exit(1);
        }
        else {
            // ---- PARENT PROCESS ----
            printf("[PARENT] My PID = %d, created child PID = %d\n", getpid(), pid);
            wait(NULL);
        }
    }

    printf("Exiting shell...\n");
    return 0;
}
```

```
jakka_bhanu@LAPTOP-DCEV: ~$ cat myshell.c
return 0;
}
-bash: syntax error near unexpected token '('
Command 'int' not found, but there are 33 similar ones.
-bash: syntax error near unexpected token '('
-bash: syntax error near unexpected token '{'
args[i++]: command not found
-bash: syntax error near unexpected token '('
-bash: syntax error near unexpected token '}'
args[i]: command not found
-bash: syntax error near unexpected token '}'
-bash: syntax error near unexpected token '('
Command 'char' not found, did you mean:
  command 'chr' from snap chr (0.80.4)
  command 'shar' from deb sharutils (1:4.15.2-13)
  command 'phar' from deb php8.5-cli (8.5.4-0ubuntu1.1)
  command 'phar' from deb php-cli (2:8.5+99ubuntu1)
  command 'chr' from deb chr (0.1.80-1)
  command 'chr' from deb chr-tiny (0.1.80-1)
  command 'car' from deb ucommon-utils (7.0.1-1)
  command 'chat' from deb ppp (2.5.2-1+1.2)
  command 'pchar' from deb pchar (1.5-7)
See 'snap info <snapname>' for additional versions.
Command 'char' not found, did you mean:
  command 'chr' from snap chr (0.80.4)
  command 'pchar' from deb pchar (1.5-7)
  command 'chr' from deb chr (0.1.80-1)
  command 'chr' from deb chr-tiny (0.1.80-1)
  command 'car' from deb ucommon-utils (7.0.1-1)
  command 'chat' from deb ppp (2.5.2-1+1.2)
  command 'phar' from deb php8.5-cli (8.5.4-0ubuntu1.1)
  command 'phar' from deb php-cli (2:8.5+99ubuntu1)
  command 'shar' from deb sharutils (1:4.15.2-13)
```

```
jakka_bhanu@LAPTOP-DCEV: ~$ ./myshell
-bash: syntax error near unexpected token 'args[0],'
-bash: syntax error near unexpected token '{'
-bash: syntax error near unexpected token 'stderr,'
-bash: syntax error near unexpected token '}'
-bash: syntax error near unexpected token 'args[1]'
-bash: syntax error near unexpected token '"cd failed"'
-bash: syntax error near unexpected token '}'
-bash: syntax error near unexpected token '}'
-bash: continue: only meaningful in a 'for', 'while', or 'until' loop
-bash: syntax error near unexpected token '}'
-bash: syntax error near unexpected token '('
-bash: syntax error near unexpected token '{'
-bash: syntax error near unexpected token '"fork failed"'
-bash: syntax error near unexpected token '}'
-bash: syntax error near unexpected token 'else'
-bash: //: Is a directory
-bash: syntax error near unexpected token '"[CHILD] PID = %d, Parent PID = %d, about to run '%s'\n",'
-bash: syntax error near unexpected token ','
-bash: syntax error near unexpected token 'args[0],'
-bash: //: Is a directory
-bash: syntax error near unexpected token '"exec failed"'
-bash: syntax error near unexpected token '1'
-bash: syntax error near unexpected token '}'
-bash: syntax error near unexpected token 'else'
-bash: //: Is a directory
-bash: syntax error near unexpected token '"[PARENT] My PID = %d, created child PID = %d\n",'
-bash: syntax error near unexpected token 'NULL'
-bash: syntax error near unexpected token '}'
-bash: syntax error near unexpected token '}'
-bash: syntax error near unexpected token '"Exiting shell...\n"'
-bash: return: can only 'return' from a function or sourced script
-bash: syntax error near unexpected token '}'
jakka_bhanu@LAPTOP-DCEV: ~$ ./myshell
```

```
jakka_bhanu@LAPTOP-DCEV: x  Ubuntu x + v
-bash: syntax error near unexpected token `}'
jakka_bhanu@LAPTOP-DCEVSCQ0:~/skill1$ ./myshell
myshell> pwd
/home/jakka_bhanu/skill1
myshell> ls
Makefile README.md include myshell obj src
myshell> exit
jakka_bhanu@LAPTOP-DCEVSCQ0:~/skill1$ cat src/main.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_ARGS 64

void parse_input(char *input, char **args) {
    int i = 0;
    char *token = strtok(input, " ");
    while (token != NULL && i < MAX_ARGS - 1) {
        args[i++] = token;
        token = strtok(NULL, " ");
    }
    args[i] = NULL;
}

int main() {
    char input[1024];
    char *args[MAX_ARGS];

    printf("Shell started. My PID = %d, Parent (terminal) PID = %d\n", getpid(), getppid());

    while (1) {
```

```
jakka_bhanu@LAPTOP-DCEV: x  Ubuntu x + v

        while (1) {
            printf("myshell> ");
            fflush(stdout);

            if (fgets(input, sizeof(input), stdin) == NULL) {
                printf("\n");
                break;
            }

            input[strcspn(input, "\n")] = '\0';

            if (strlen(input) == 0) {
                continue;
            }

            if (strcmp(input, "exit") == 0) {
                break;
            }

            parse_input(input, args);

            if (args[0] == NULL) {
                continue;
            }

            if (strcmp(args[0], "cd") == 0) {
                if (args[1] == NULL) {
                    fprintf(stderr, "cd: expected argument\n");
                } else {
                    if (chdir(args[1]) != 0) {
                        perror("cd failed");
                    }
                }
            }
        }
```



```
jakka_bhanu@LAPTOP-DCEV: x  Ubuntu x + v
// execvp only returns if it failed
perror("exec failed");
exit(1);
}
else {
    // ---- PARENT PROCESS ----
    printf("[PARENT] My PID = %d, created child PID = %d\n", getpid(), pid);
    wait(NULL);
}
}

printf("Exiting shell...\n");
return 0;
}

jakka_bhanu@LAPTOP-DCEVSCQ0:~/skill1$ rm -rf obj myshell
jakka_bhanu@LAPTOP-DCEVSCQ0:~/skill1$ make
gcc -Wall -g -Iinclude src/main.c -o myshell
jakka_bhanu@LAPTOP-DCEVSCQ0:~/skill1$ ./myshell
Shell started. My PID = 832, Parent (terminal) PID = 348
myshell> pwd
[PARENT] My PID = 832, created child PID = 835
[CHILD] PID = 835, Parent PID = 832, about to run 'pwd'
/home/jakka_bhanu/skill1
myshell> exit
Exiting shell...
jakka_bhanu@LAPTOP-DCEVSCQ0:~/skill1$ Shell started. My PID = 832, Parent (terminal) PID = 348 ← your shell itself, child of the terminal
[PARENT] My PID = 832, created child PID = 835 ← shell (832) forks a new child (835)
[CHILD] PID = 835, Parent PID = 832, about to run 'pwd' ← child confirms 832 is its parent
/home/jakka_bhanu/skill1 ← actual pwd output, run via execvp()
-bash: syntax error near unexpected token '('
-bash: syntax error near unexpected token '('
[CHILD]: command not found
```

```
jakka_bhanu@LAPTOP-DCEV: x  Ubuntu x + v
jakka_bhanu@LAPTOP-DCEVSCQ0:~/skill1$ ./myshell &
[1] 870
Shell started. My PID = 870, Parent (terminal) PID = 348
myshell>
[1]+  Stopped                  ./myshell
jakka_bhanu@LAPTOP-DCEVSCQ0:~/skill1$ pstree -p $(pgrep -f myshell | head -1)
myshell(870)
jakka_bhanu@LAPTOP-DCEVSCQ0:~/skill1$ sudo apt install psmisc -y
[sudo: authenticate] Password:
psmisc is already the newest version (23.7-2ubuntu2).
psmisc set to manually installed.
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 12
jakka_bhanu@LAPTOP-DCEVSCQ0:~/skill1$ fg
./myshell
exit
Exiting shell...
jakka_bhanu@LAPTOP-DCEVSCQ0:~/skill1$ ./myshell
Shell started. My PID = 972, Parent (terminal) PID = 348
myshell> sleep 5
[PARENT] My PID = 972, created child PID = 978
[CHILD] PID = 978, Parent PID = 972, about to run 'sleep'
myshell> ./myshell <<< '$sleep 6\nexit' & sleep 2 && pstree -p $(pgrep -f myshell | head -1) && ps --forest -o pid,ppid,cmd -g $(pgrep -f myshell | head -1)
[PARENT] My PID = 972, created child PID = 1019
[CHILD] PID = 1019, Parent PID = 972, about to run './myshell'
Shell started. My PID = 1019, Parent (terminal) PID = 972
myshell> exit
Exiting shell...
myshell> exit
Exiting shell...
jakka_bhanu@LAPTOP-DCEVSCQ0:~/skill1$ ./myshell <<< '$sleep 6\nexit' & sleep 2 && pstree -p $(pgrep -f myshell | head -1) && ps --forest -o pid,ppid,cmd -g $(pgrep -f myshell | head -1)
```

```

jakka_bhanu@LAPTOP-DCEV: x  Ubuntu  x  +  v
[PARENT] My PID = 972, created child PID = 1019
[CHILD] PID = 1019, Parent PID = 972, about to run './myshell'
Shell started. My PID = 1019, Parent (terminal) PID = 972
myshell> exit
Exiting shell...
myshell> exit
Exiting shell...
jakka_bhanu@LAPTOP-DCEVSCQ0:~/skill1$ ./myshell <<< '$sleep 6\nexit' & sleep 2 && pstree -p $(pgrep -f myshell | head -1) && ps --for
est -o pid,ppid,cmd -g $(pgrep -f myshell | head -1)
[1] 1045
Shell started. My PID = 1045, Parent (terminal) PID = 348
myshell> [PARENT] My PID = 1045, created child PID = 1047
[CHILD] PID = 1047, Parent PID = 1045, about to run 'sleep'
myshell(1045)---sleep(1047)
PID      PPID CMD
jakka_bhanu@LAPTOP-DCEVSCQ0:~/skill1$ strace -f -o shell_trace.txt ./myshell <<< '$pwd\nexit'./myshell <<< '$pwd\nexit'
Shell started. My PID = 1073, Parent (terminal) PID = 1071
myshell> [PARENT] My PID = 1073, created child PID = 1074
[CHILD] PID = 1074, Parent PID = 1073, about to run 'pwd'
/home/jakka_bhanu/skill1
myshell> Exiting shell...
[1]+  Done                  ./myshell <<< 'sleep 6
exit'
jakka_bhanu@LAPTOP-DCEVSCQ0:~/skill1$ grep -E "fork|clone|exec|wait" shell_trace.txt
1073  execve("./myshell", ["/myshell"], 0x7ffe146ea298 /* 26 vars */) = 0
1073  clone(child_stack=NULL, flags=CLONE_CHILD_CLEARTID|CLONE_CHILD_SETTID|SIGCHLD, child_tidptr=0x7f9bbeb36a10) = 1074
1073  wait4(-1 <unfinished ...>
1074  execve("/usr/local/sbin/pwd", ["pwd"], 0x7ffe80e29178 /* 26 vars */) = -1 ENOENT (No such file or directory)
1074  execve("/usr/local/bin/pwd", ["pwd"], 0x7ffe80e29178 /* 26 vars */) = -1 ENOENT (No such file or directory)
1074  execve("/usr/sbin/pwd", ["pwd"], 0x7ffe80e29178 /* 26 vars */) = -1 ENOENT (No such file or directory)
1074  execve("/usr/bin/pwd", ["pwd"], 0x7ffe80e29178 /* 26 vars */) = 0
1073  <... wait4 resumed>, NULL, 0, NULL) = 1074
jakka_bhanu@LAPTOP-DCEVSCQ0:~/skill1$

```